



Swift6 마이그레이션 전 마음의 준비 하는 법 🧘

헤린 | 리나 | 병호



목차

1

Swift6...
도대체 뭐가
바뀐거야?

Swift6의 핵심

기존 Swift를 사용한 개발 과정에서 발생할 수 있는 문제점에 대해 알아보고 Swift6는 이 문제를 어떻게 해결하고자 했는지 알아봅시다 !

필요한 사전지식

Swift6를 도입하기 전 어떤 개념들을 알아야하는지 알아봅시다!

2

숨쉬..숨쉬..
Swift6
마이그레이션
해보기 🧘

점진적인 마이그레이션 방법 소개

무작정 Swift6으로 올리면 되는거 아니냐구요?
No...

실습 케이스 공유

실습해보면서 느낀 점

Swift6...
도대체 뭐가 바뀐거야?

Swift6... 도대체 뭐가 바뀐거야?

Swift6의 핵심

keyword 

Data Race

- 여러 스레드가 동시에 동일한 메모리 위치에 접근하고, 그 중 하나 이상이 쓰기 작업을 수행할 때 발생하는 문제
- 이로 인해 예측할 수 없는 결과, 데이터 손상, 충돌 등이 발생할 수 있음

// Data Race 발생 가능한 코드 예시

```
class Counter {  
    var count = 0  
  
    func increment() {  
        count += 1 // 이 부분이 원자적 연산이 아님  
    }  
}
```

```
let counter = Counter()  
// 여러 스레드에서 동시에 호출  
DispatchQueue.concurrentPerform(iterations: 1000) { _ in  
    counter.increment()  
}  
// 결과가 1000이 아닐 수 있음
```

Swift6... 도대체 뭐가 바뀐거야?

Swift6의 핵심

**Swift6 이전 개발 시
Data Race 어떻게 방지할 수 있었지 ?!**

- ➔ NSLock, Semaphore 등을 사용해서 개발자가 직접 방지할 수 있도록 코드를 구현해야함.
즉 멀티스레드 환경에서 발생하는 Data Race 문제는 여전히 개발자의 책임으로 남아있었음
- ➔ 하지만 Data Race는 동시성 코드에서 발생하는 가장 심각하고 찾기 어려운 버그 유형 중 하나 .
디버깅도 어려운 경우가 많다 ...
- ➔ 개발자의 책임을 덜어낼 수 있는 방법이 있을까?

Swift6... 도대체 뭐가 바뀐거야?

Swift6의 핵심

- ✓ Swift6부터는 Data Race가 발생할 수 있는 상황을 컴파일 타임에 최대한 체크하자!!
- ✓ 이전 버전에서 도입된 `async/await`, Actor 등의 기능을 더욱 발전시켜 완전한 동시성 모델을 제공

Swift6... 도대체 뭐가 바뀐거야?

필요한 사전지식

Task

any/some Keyword

async await

Continuation

actor hop

Modern Concurrency

atomicity

Main Actor

nonisolated(unsafe)

@preconcurrency

Actor

nonisolated

Sendable

re-entrancy

Global Actor

sending

@unchecked Sendable

Actor isolation

Non-sendable

Swift6... 도대체 뭐가 바뀐거야?

필요한 사전지식: Actor

- ✓ 자신의 상태(프로퍼티)에 대한 독점적인 접근을 보장하는 참조 타입
- ✓ 여러 스레드가 동시에 같은 데이터에 접근하는 것을 방지하여 데이터 무결성을 유지함

특징 1. 격리된 상태(isolation)

actor 내부의 프로퍼티는 actor 외부에서 직접 접근할 수 없고, actor 메서드를 통해서만 접근 가능함

특징 2. 비동기 함수 호출

actor의 메서드는 기본적으로 비동기적으로 실행됨. 외부에서 actor의 메서드를 호출할 때는 await 키워드를 사용해야 함.

특징 3. 직렬화된 접근

actor는 자신의 상태에 대한 접근을 직렬화하여, 한 번에 하나의 작업만 실행되도록 보장함.

Swift6... 도대체 뭐가 바뀐거야?

필요한 사전지식: Sendable

- ✓ 타입 안전성을 보장하기 위한 프로토콜
- ✓ Sendable 프로토콜을 채택하면 동시성 영역 간에 안전하게 전달될 수 있음을 컴파일러에게 알려줌

특징 1. 동시성 경계 안전성

Sendable 타입은 스레드나 태스크 간에 안전하게 공유될 수 있음

특징 2. 값 타입 자동 준수

구조체, 열거형과 같은 값 타입은 모든 프로퍼티가 Sendable을 준수한다면 자동으로 Sendable을 준수함

특징 3. 참조 타입 제한

클래스와 같은 참조 타입은 특별한 조건(불변성 보장 등)을 만족해야만 Sendable을 준수할 수 있음

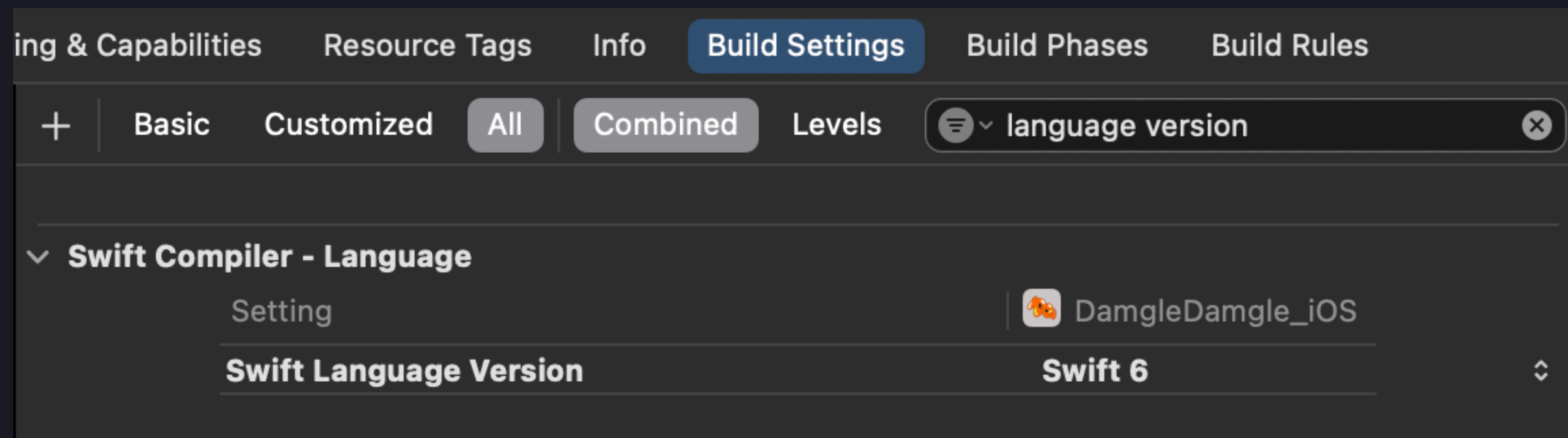
숨쉬..숨쉬.. Swift6
마이그레이션 해보기 🧘

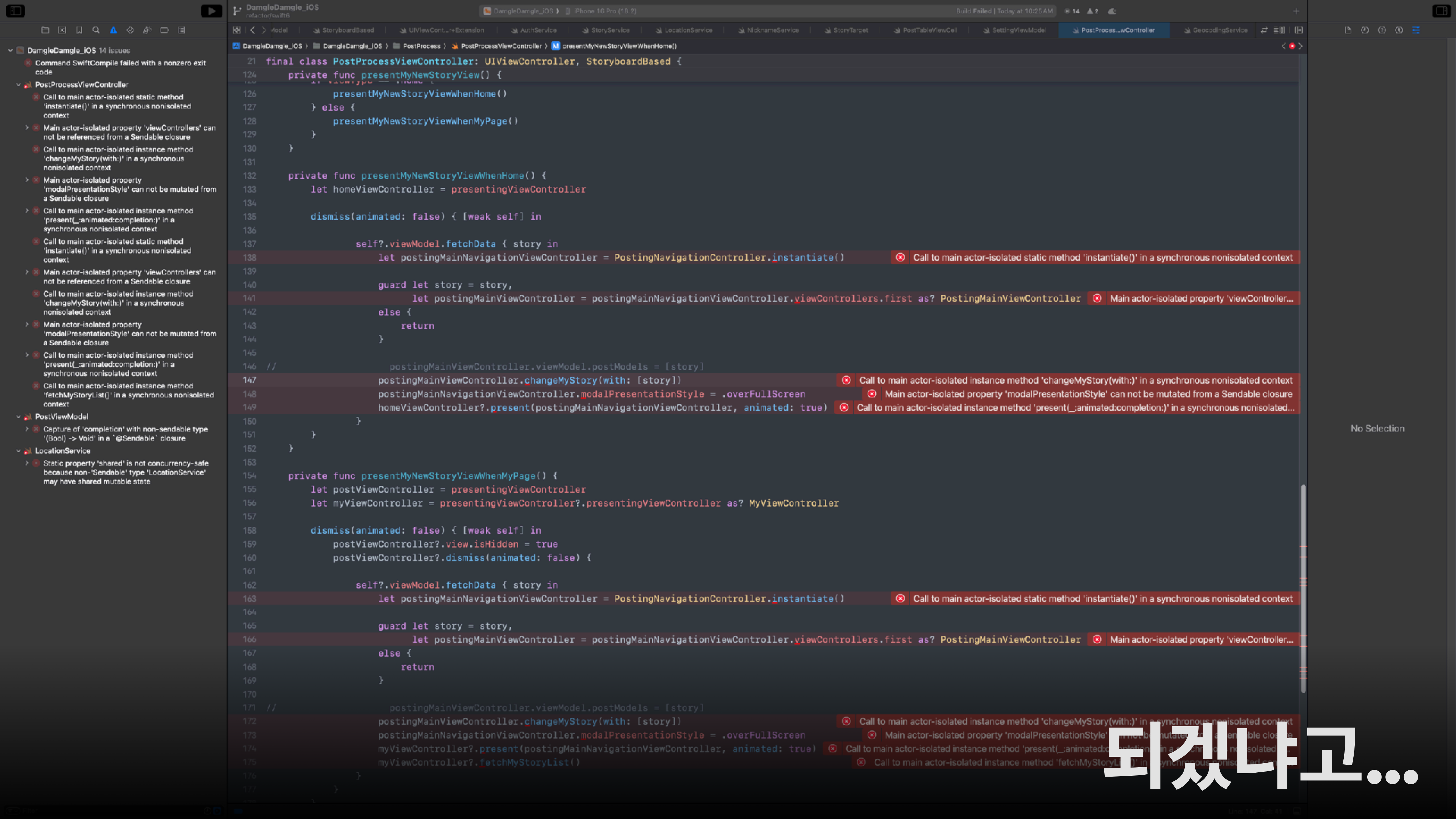
Swift6 마이그레이션 해보기 🧘

점진적인 마이그레이션 방법 소개

직장동료👤: Swift6로 마이그레이션 준비해볼까요?

병호🤸: 네네 그래요~~ 그럼 일단 Swift Language Version부터 올려볼까~~~?





Swift6 마이그레이션 해보기 🧘

점진적인 마이그레이션 방법 소개

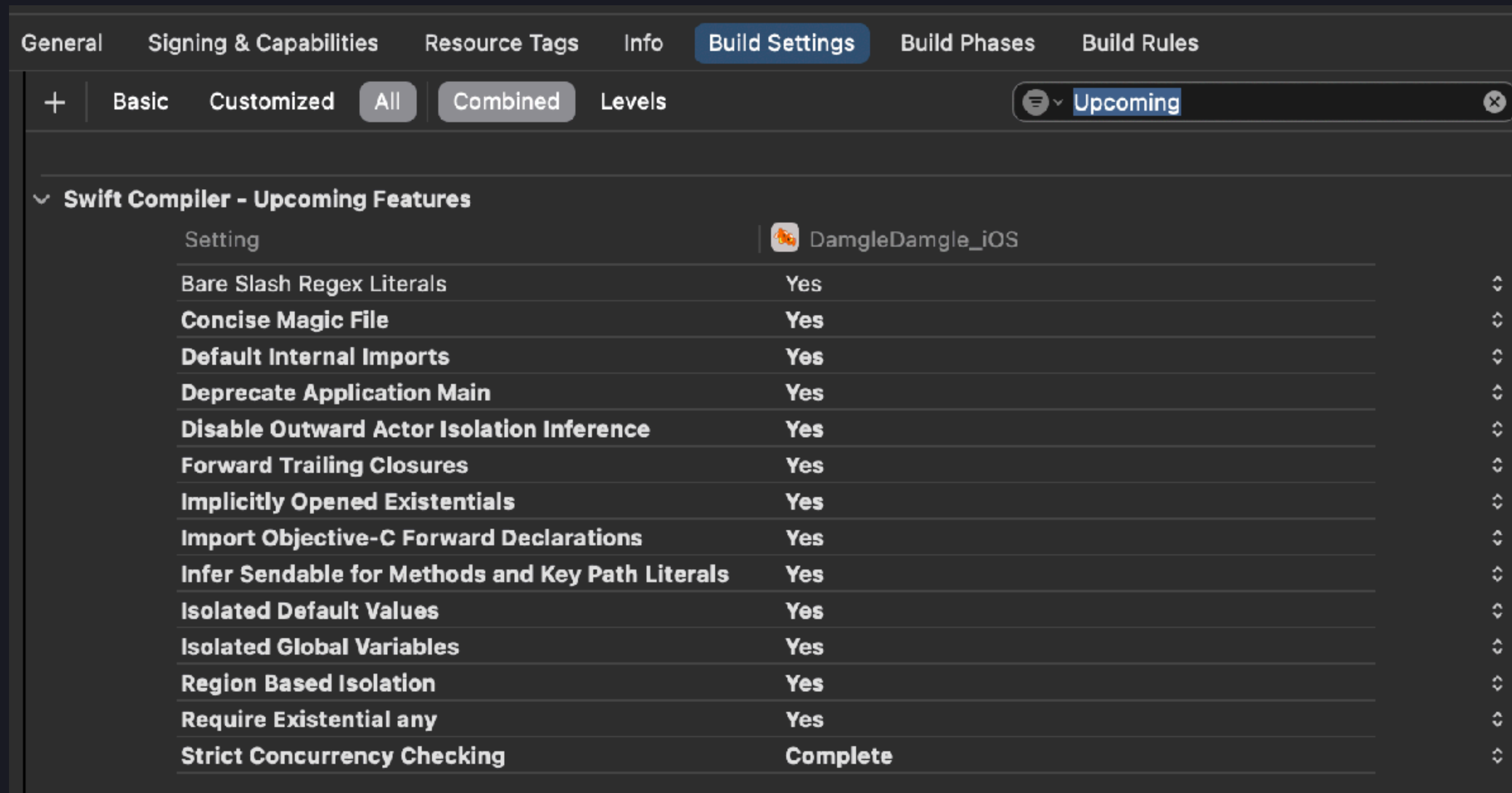
물론! 차근차근 하나하나 고치고 빌드 고치고 빌드 500번 정도 하면 다 고칠 수 있겠지만...
이 방법보다 Swift6 마이그레이션은 점진적으로 도입하는걸 추천합니다 !

어떻게?

Upcoming Feature를 사용해서!

Swift6 마이그레이션 해보기 🧘

점진적인 마이그레이션 방법 소개

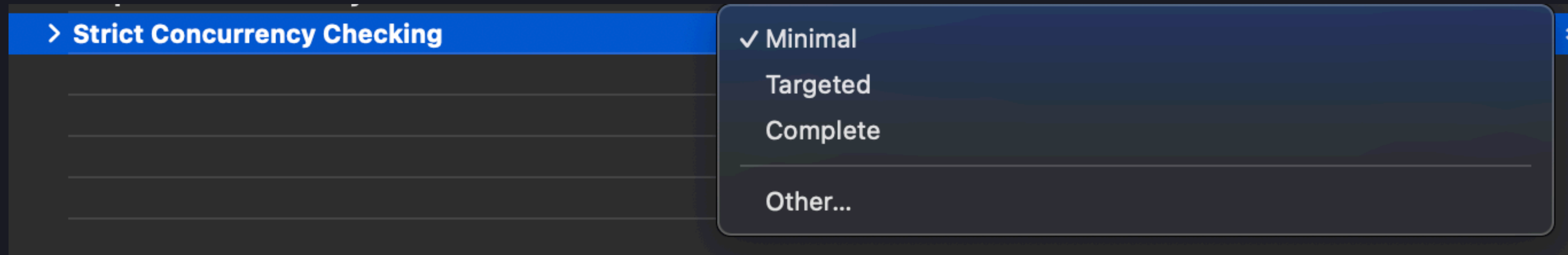


Build Settings - Upcoming Feature 검색

➡ 새로 개발된 기능들을 실험적으로 임시 적용해볼 수 있는 기능

Swift6 마이그레이션 해보기 🧘

점진적인 마이그레이션 방법 소개



이 중 **Strict Concurrency Checking** 단계를 변경해서
Swift5를 유지한 상태로 Swift6에서 뜰 오류들을 에러가 아닌 경고로 미리 대응할 수 있음 !

- ✓ **Minimal:** Swift 5.5~5.8 시절처럼 **기본적인 동시성 오류**만 탐지합니다.
- ✓ **Targeted:** @Sendable, @MainActor, nonisolated, @preconcurrency 등 **명시적으로 동시성을 선언한 영역**에 대해서만 검사합니다.
- ✓ **Complete:** 전체 검사 수행. 동시성 관련 모든 경고 및 오류 노출

Swift6 마이그레이션 해보기 🧘

실습 케이스 공유

1 Sendable & 전역변수 관련 오류

```
final class UserManager {  
    static let shared: UserManager = UserManager()  
    private init() {}  
}
```

- UserManager는 User와 관련된 accessToken, refreshToken 등을 관리하는 객체
- 싱글톤 패턴으로 사용하고 있음

Swift6 마이그레이션 해보기 🧘

실습 케이스 공유

```
final class UserManager {  
    static let shared: UserManager = UserManager()  
    private init() {}  
}
```

error ⚠

Static property 'shared' is not concurrency-safe because non-'Sendable' type 'UserManager' may have shared mutable state; this is an error in the Swift 6 language mode

➡ UserManager가 Sendable 하지 않는데 shared로 전역 인스턴스임

UserManager가 Sendable 하지 않는데 shared로 전역 인스턴스임

Swift6 마이그레이션 해보기 🧘

실습 케이스 공유

해결 방법 1)

만약 UserManager 객체가 메인 액터에서만 접근이 돼야한다면 shared 앞에 @MainActor를 붙여라

```
@MainActor  
final class UserManager
```

🚫 근데 UserManager는 accesstoken과 같이 유저와 관련된 값을 관리하는 객체이기 때문에 메인액터를 붙이는건 적합한 해결방법은 아니라고 판단함

Swift6 마이그레이션 해보기 🧘

실습 케이스 공유

해결 방법 2)

UserManager를 Actor로 변경

`actor UserManager`

- ✅ refreshToken, accessToken 등 데이터를 관리하는 객체이기 때문에 이론상 actor로 안정성을 보장하는 것이 좋음!
- 🚫 근데 마이그레이션 직접 진행해보니 UserManager 내부 프로퍼티 혹은 메서드에 접근하는 모든 부분에 await 키워드가 필요함
 - 즉 호출부가 비동기 환경이어야함
 - 하지만 현재 구조상 비동기 환경을 모두 구축해주기가 어려운 부분이 있음(callback 구조라던지...)
 - 참부터 개발을 잘하자.....^^

Swift6 마이그레이션 해보기 🧘

실습 케이스 공유

회피 방법 1)

만약 외부로부터 안전하게 보호되고 있다면 concurrency-safety 확인을 disable시켜라
→ 대상이 Class 전체인 경우

```
final class UserManager: @unchecked Sendable
```

✓ @unchecked Sendable을 사용해서 class 전체에 대한 검사를 진행하지 않을 수도 있음!
하지만 회피대상이 class 전체라 범위가 다소 넓다는 단점이 있음

Swift6 마이그레이션 해보기 🧘

실습 케이스 공유

회피 방법 2)

만약 외부로부터 안전하게 보호되고 있다면 concurrency-safety 확인을 disable시켜라
→ 대상이 shared 프로퍼티인 경우

```
nonisolated(unsafe) static let shared: UserManager = UserManager()
```

✅ nonisolated(unsafe)를 사용해서 shared 프로퍼티에 대한 검사만 진행하지 않도록 함.

Swift6 마이그레이션 해보기 🧘

실습 케이스 공유

2 또다른 Sendable 관련 오류

```
extension HomeViewController: NMFMapViewCameraDelegate {
    func mapViewCameraIdle(_ mapView: NMFMapView) {
        viewModel.getStoryFeed { result in
            switch result {
            case .success(let homeModel):
                guard let homeModel = homeModel else { return }
                self.addMarker(homeModel: homeModel)
            case .failure(let error):
                self.showAlertController(
                    type: .single,
                    title: "오류 발생",
                    message: error.localizedDescription,
                    okActionTitle: "확인"
                )
            }
        }
    }
}
```

- 지도 반경 변경 시 해당 반경에 해당하는 story를 받아오는 ViewController 내부 코드
- 현재 callback 구조로 구현돼있는 viewModel.getStoryFeed 메서드를 continuation을 사용하여 modern concurrency 방식으로 변경하려고 함!

Swift6 마이그레이션 해보기 🧘

실습 케이스 공유

2 또다른 Sendable 관련 오류

```
func mapViewCameraIdle(_ mapView: NMFMapView) {  
    Task {  
        let result = await self.viewModel.getStoryFeed()  
    }  
}
```

➡ await 메서드를 사용하기 위해 Task를 생성해야함

Swift6 마이그레이션 해보기 🧘

실습 케이스 공유

2 또다른 Sendable 관련 오류

error ⚠️

Non-sendable type 'Result<HomeModel?, any Error>' returned by implicitly asynchronous call to nonisolated function cannot cross actor boundary; this is an error in the Swift 6 language mode

➡ self.viewModel이 Sendable이 아니라서 actor 경계를 건널 수 없어서 에러남

Swift6 마이그레이션 해보기 🧘

실습 케이스 공유

2 또다른 Sendable 관련 오류

```
func mapViewCameraIdle(_ mapView: NMFMapView) {  
    let viewModel = self.viewModel  
  
    Task {  
        let result = await viewModel.getStoryFeed()  
    }  
}
```

✅ 이런 경우 Task 진입 전에 viewModel을 미리 캡처하고 들어가면 우회할 수 있다!

Swift6 마이그레이션 해보기 🧘

실습 케이스 공유

3 Region Based Isolation 관련 오류

컴파일러가 Sendable이 아닌 값이 안전하게 다른 격리 영역으로 전달될 수 있는지 정밀하게 분석하는 "격리 영역(isolation regions)" 개념에 대한 오류

Swift6 마이그레이션 해보기 🧘

실습 케이스 공유

```
DispatchQueue.main.async { [weak self] in
    self?.appearedToasts.reversed().forEach { self?.dismiss(toastView: $0) }
    self?.present(toastView: toastView, seconds: seconds)
}
```

⚠️ Sending 'self' risks c

ⓘ Task-isolated 's

Task-isolated 'self' is captured by a main actor-isolated closure. main actor-isolated uses in closure may race against later nonisolated uses

➡ Task-isolated된 'self'가 @MainActor로 격리된 클로저에 의해 캡처되었는데, 이 클로저 내에서 MainActor-isolated된 사용이 이후의 non-isolated된 사용과 경쟁 상태(race condition)가 발생할 수 있다는 의미

Swift6 마이그레이션 해보기 🧘

실습 케이스 공유

해결 방법 1)

해당 메서드나 프로퍼티에 `nonisolated` 키워드를 사용하여 액터 격리에서 제외

```
nonisolated private func show(message: String?, seconds: Double) {  
    guard let message = message else {  
        return  
    }  
  
    let toastView = ToastView(frame: .zero)  
    ...  
}
```

해당 메서드 안에서 `ToastView`를 생성하고 있는데 `ToastView`는 `UIView`의 subclass라서 자동으로 Main Actor로 격리되고 있는 상황

- 🚫 즉 비격리 메서드에서 Main Actor의 메서드 호출 → 비동기(`await` 키워드)로 호출해야함
→ 하지만 현재 메서드는 비동기 환경이 아님... → 이걸 반영하기 위해서 너무 많은 코드가 변경돼야함

Swift6 마이그레이션 해보기 🧘

실습 케이스 공유

해결 방법 2)

@MainActor 속성을 클래스/구조체 레벨에 적용하여 모든 멤버가 메인 액터에서 실행되도록 보장

```
@MainActor  
final class Toast: NSObject {
```

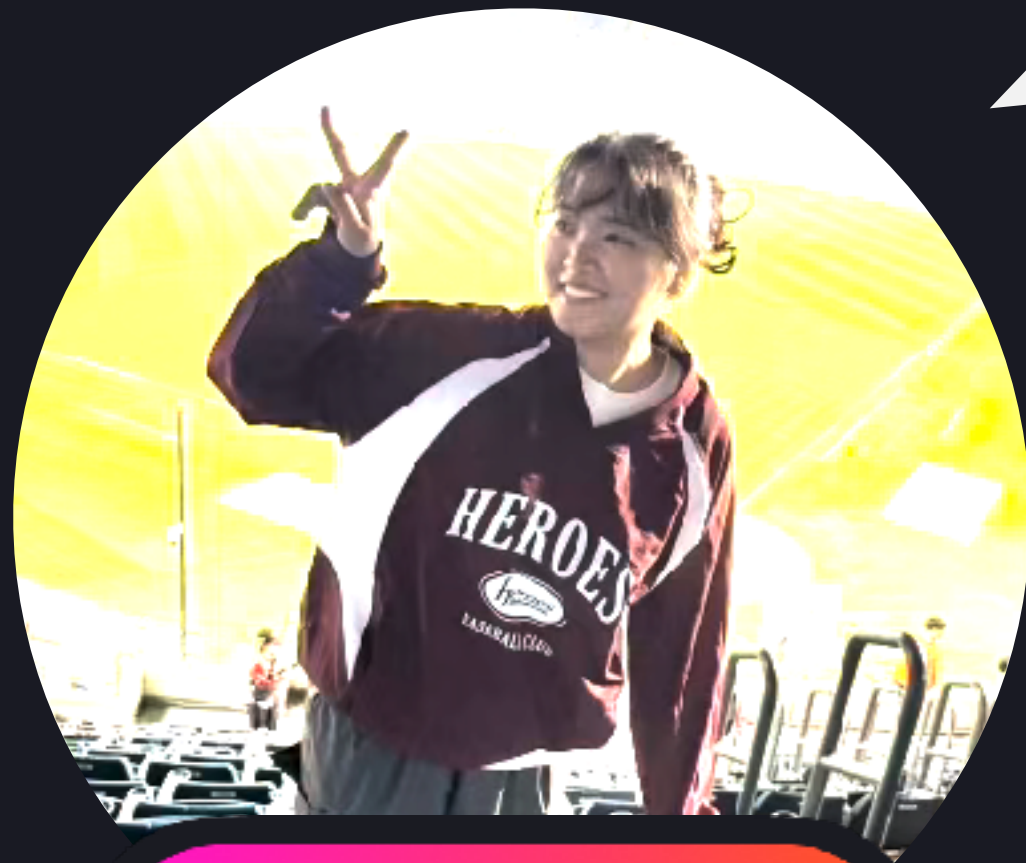
- ✅ Toast도 결국은 UI컴포넌트를 보여주고 숨기는 등 UI와 관련된 로직을 처리하는 부분이기 때문에 Main Actor로 격리하는 것도 좋은 방법이라고 판단했음!

그럼 자연스럽게 앞에서 작성한 DispatchQueue.Main.async 코드도 작성하지 않아도 되는 상황이 됨

Swift6 마이그레이션 해보기 🧘

실습해보면서 느낀 점

- 💤 일단 callback closure 기반 코드를 continuation을 사용한 새로운 API를 적용하든 modern concurrency로 리팩토링 해두는게 가장 중요한 것 같당...
그거 때문에 해결하기 복잡해지는 코드가 제일 많아지는것같은 느낌적인 느낌 ㅠ
- 💤 작은 사이드플젝 이거 화면 몇개지 5개? 이런 플젝도 이렇게 오류가 많이 뜨는데 회사꺼는 도대체 얼마나 난장판일까? ㅋㅋ...



헤린

Swift6 마이그레이션 해보기 🧘

실습해보면서 느낀 점

- 😴 만능 키처럼 @MainActor 붙이거나, 그냥 컴파일에서 무시하게 할 수 있는데.. ㅎㅎ 결과적으로 이들이 원하는건.. 제대로 된 컨커런시를 다시 구축하게 만드는듯^^
- 😴 처음에 뜬 경고가 다가 아님^ㅅ^ 수정할수록 번식함.. 버그가 버그를 부른다



리나

Swift6 마이그레이션 해보기 🧘

실습해보면서 느낀 점

- 😴 컴파일 단계에서 데이터 레이스를 잡아주는 것은 아주 유용한 것 같다.
- 😴 의도는 좋았으나 왜 이제 와서 내놓는 것인지!!! 그동안 코드 잘못 짤..ㅠ



병호



Mash-Up
iOS Team

2025.05.13

Thank you.

